Using Uproot to Rediscover the Higgs Boson in ATLAS Open Data

Oliver Alban

May 18, 2026

1 Overview

This exercise is aimed towards those interested in experimental high-energy physics with limited programming experience. Readers will be introduced to Python and the needed packages (such as Uproot and Numpy) to complete three activities:

1. Use real data to plot the masses of di-lepton systems to observe a peak at the mass of the Z boson
2. Use simulated data to observe a peak at the mass of the Higgs boson
3. Use real data to observe a peak at the mass of the Higgs boson

Completion of these activities will require knowledge in relevant physics topics such as fundamental particles, decay modes, pseudorapidity, and 4-vector calculations. An introduction to each of these topics is included in section 2 for readers who may not be familiar.

This exercise also requires a basic understanding of Python as well as the modules needed for data analysis. An introduction to the necessary information and resources to learn Python are included in Section 3.

You will be working with an open release of data collected by the ATLAS detector operated by CERN (Center for European Nuclear Research) a global collaboration to advance high-energy physics located in Geneva, Switzerland. ATLAS stands for *A Toroidal LHC Apparatus*; it is one of several detectors at the Large Hadron Collider (LHC). The LHC accelerates beams of protons to near the speed of light to produce high-energy collisions recorded by ATLAS. The detector includes several components, such as liquid argon calorimeters and a silicon pixel detector among many others that record a wide variety of information about the particles that pass through it. The ATLAS detector is most well known for being one of the primary detectors used to prove the existence of the Higgs boson. [2] You will be using ATLAS open data to see for yourself the existence of the Higgs boson.

2 Physics Context

2.1 Mass and 4-Vectors

4-vectors (also known as *Lorentz vectors*) are vectors used in relativistic calculations. They are defined by their ability to undergo *Lorentz transformations.* They have one *time* and three *space* components, as can be seen plainly in the *4-position* which records a location in *spacetime*.

$$\vec{S} = \begin{pmatrix} t \\ x \\ y \\ z \end{pmatrix} \tag{1}$$

Notice the apparent inconsistency in the units of the components of $\vec{S}$: t has units of time while x, y, and z have units of length. In actuality, there is a hidden factor of the speed of light (c) such that the *4-position* is truly:

$$\vec{S_1} = \begin{pmatrix} ct \\ x \\ y \\ z \end{pmatrix} \tag{2}$$

As can be seen, all components have units of length. However, these factors of c become numerous quickly, so physicists often simplify their calculations with a system of *natural units* where $c \equiv 1$. With this definition, no unit contradiction exists in $\vec{S}$ and t now has units of length.

The 4-vectors that you will work with to rediscover the Higgs boson are *4-momentums* .[1] [2]

$$\vec{P} = \begin{pmatrix} E \\ p_x \\ p_y \\ p_z \end{pmatrix} \tag{3}$$

For the *4-momentum*, the energy *(E)* is the time component and the space components are the components of the *3-momentum.*[3] High-energy physicists measure both energy and momentum in terms of gigaelectron-volts (GeV).

The scalar product of two 4-vectors acts a little differently than other vectors. All the spacelike quantities pick up a minus sign like so:

$$\vec{A} \cdot \vec{B} = \begin{pmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \end{pmatrix} \cdot \begin{pmatrix} B_0 \\ B_1 \\ B_2 \\ B_3 \end{pmatrix} = A_0 B_0 - A_1 B_1 - A_2 B_2 - A_3 B_3 \tag{4}$$

---

[1] $\vec{P}$ can be derived by differentiating $\vec{S}$ with respect to the proper time τ (taking note that $\gamma\, d\tau = dt$) and then multiplying by m.

[2] If you neglect to use natural units, the 4-momentum will be $\begin{pmatrix} \frac{E}{c} & p_x & p_y & p_z \end{pmatrix}$

[3] The *3-momentum* is simply the non-relativistic momentum: $\mathbf{p} = \begin{pmatrix} p_x & p_y & p_z \end{pmatrix}$

From this we can see that a 4-vector squared is:

$$(\vec{A})^2 = \vec{A} \cdot \vec{A} = (A_0)^2 - (A_1)^2 - (A_2)^2 - (A_3)^2 \tag{5}$$

This quantity, the square of a 4-vector, is called the *invariant quantity* because it is invariant under Lorentz transformations. This means that if we were to look at a particle in different inertial frames moving with respect to each other, then write out the 4-vectors of the particle from each frame, and finally calculate the invariant quantity, we would end up with the same value every time.

The invariant quantity of the 4-momentum is:

$$(\vec{P})^2 = E^2 - p_x^2 - p_y^2 - p_z^2 = E^2 - \mathbf{p} \cdot \mathbf{p} \tag{6}$$

The mass of an object is not the "amount of stuff" that is in it, but rather its energy at rest.[4] Note that mass in natural units is not measured in kilograms but rather in gigaelectron-volts just like momentum and energy. At rest, the 4-momentum of a particle is:[5]

$$\vec{P}_{rest} = \begin{pmatrix} m \\ 0 \\ 0 \\ 0 \end{pmatrix} \tag{7}$$

And its square is:

$$(\vec{P})^2 = m^2 - 0 - 0 - 0 = m^2 \tag{8}$$

Since the square of a 4-vector is an invariant quantity, it is true in all frames, so the mass of any particle, given its 4-momentum is:

$$m = \sqrt{E^2 - \mathbf{p} \cdot \mathbf{p}} \tag{9}$$

This is the relation that you will use to calculate the masses of the Z and Higgs bosons.

2.2 The Standard Model

The *Standard Model* is the leading theory describing the *fundamental particles* and the forces that allow them to interact. *Fundamental particles* are not made up of smaller particles; they are the simplest building blocks of the universe. Figure 1 displays the different types of fundamental particles. A majority of these have corresponding antimatter particles which are much like the matter particles but with opposite properties. As an example, the electron (e^-) and its antimatter equivalent, the positron (e^+), have the same mass and behavior but opposite charges.

[4] If we avoid dropping factors of c, the energy at rest is given by $E = mc^2$.

[5] Note that a particle being "at rest" simply means that it is being observed in a frame where it does not appear to be moving. 4-vectors are frame dependent so in a frame moving with respect to the particle, the particle would have an energy greater than m.

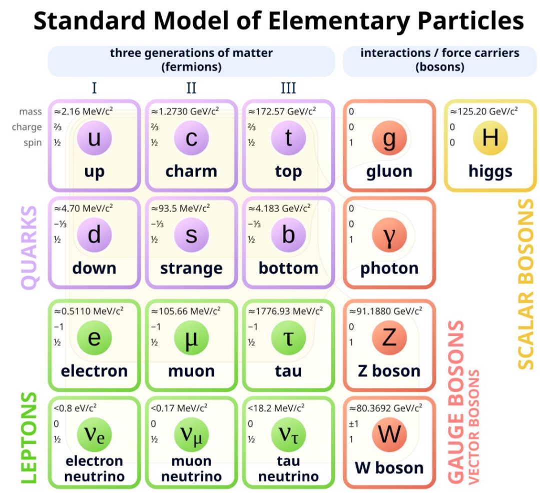

Figure 1: The fundamental particles of the standard model

In the following activities, you will be working with four different kinds of particles: electrons (e), muons (μ) (both matter and antimatter variants), Z bosons and Higgs bosons. Electrons and muons are members of a subset of particles called *leptons*. Both particles are of charge -1 (meaning their antiparticles are both of charge +1) and have a longer lifespan than the Z and Higgs Bosons. The Z boson is a neutral particle (a particle of zero charge) that mediates the *weak interaction*. The Higgs boson is a neutral particle that *couples* to other particles to give them mass.

2.3 Decay Modes

Fundamental particles can spontaneously transform into multiple other particles in a process called *particle decay*. The *decay modes* of a particle are the various ways in which a particle can decay. There are only a few decay modes that you need to know for the purpose of completing this exercise. A more complete list can be found using the *Particle Data Group*.[8]

The Z boson decays into two leptons of the same *flavor*[6] and opposite charges.[7] Thus, the two Z boson decay modes (that are relevant for this exercise) are: $Z \to e^+e^-$ and $Z \to \mu^+\mu^-$. Due to their short lifespans (the time that a particle exists before it decays), the ATLAS detector cannot directly detect Z bosons. Instead, we take advantage of the law of conservation of momentum

[6] The flavor means the type of particle. Electrons and positrons are particles of the same flavor; the same is true of muons and anti-muons.

[7] The fact that the Z boson must decay into particles of opposite charges can be derived from the law of conservation of charge. The Z is a neutral particle, so the charge of the system, before and after the decay, must be zero.

to observe them. The 4-momentum of a particle undergoing particle decay is conserved, so if we can confirm that two leptons likely came from a Z boson, the sum of the their 4-momentums is equal to the 4-momentum of the Z boson they decayed from, which we can then use to calculate the mass.

The Higgs boson decays into two Z bosons.[8]. One of these Z bosons is an *off-shell* or *virtual* Z boson, meaning it has less mass than we would expect. A typical Z boson has a mass of around 91 GeV while the off-shell Z boson is around 34 GeV.[9] The reason these off-shell Z bosons are able to exist is by borrowing the needed energy from the vacuum of space around them before quickly decaying into more stable particles. The reason they are able to do this is due to *Heisenberg's Uncertainty Principle* which sets limits on the accuracy that pairs of physical properties can be measured. The energy-time uncertainty relation is shown below:

$$\Delta E \cdot \Delta t \geq \frac{\hbar}{2} \tag{10}$$

The other Z boson is referred to as *on-shell* to signify that it has the expected amount of mass. Both Z bosons decay as described above into two leptons each. Thus, the four Higgs boson decay modes are:

1. $H \rightarrow ZZ^* \rightarrow e^+e^-e^+e^-$
2. $H \rightarrow ZZ^* \rightarrow e^+e^-\mu^+\mu^-$
3. $H \rightarrow ZZ^* \rightarrow \mu^+\mu^-e^+e^-$
4. $H \rightarrow ZZ^* \rightarrow \mu^+\mu^-\mu^+\mu^-$

The production and decay of the Higgs boson is shown in Figure 2.

Much like the Z boson, we will not be directly observing the Higgs Boson in ATLAS open data, instead, we will look for 4 leptons that meet all the characteristics of leptons which would have decayed from a Higgs boson, then we will add together their 4-vectors to obtain a 4-vector for the Higgs boson they decayed from.

2.4 Pseudorapidity

ATLAS records the momentum of the particles it detects in a special coordinate system based in *pseudorapidity*. The *pseudorapidity* (η) is defined as follows:

$$\eta = -ln[tan(\frac{\theta}{2})] \tag{11}$$

θ is the angle between the beam and the 3-momentum. As θ varies from 0 to π, η varies from ∞ to $-\infty$. The reason that we use pseudorapidity η as opposed

[8]The Higgs boson can also decay into other particles as well, such as two photons, but the 4 lepton channel is the one relevant for this exercise

[9]This is an approximate mass. The off-shell Z boson will have as much mass as is needed to make is so the Higgs boson has the appropriate amount of mass.

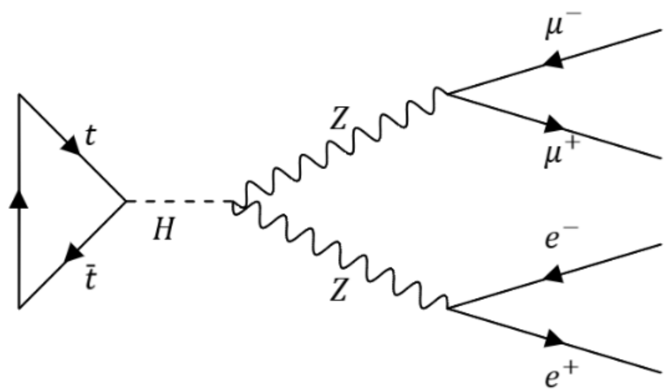

Figure 2: Decay of a Higgs boson into four leptons

to the angle θ is because η is constant under Lorentz transformations[10] while θ is not.

Detector information in pseudorapidity coordinates can be converted to Cartesian coordinates with the following formulas:

$$p_x = p_\perp \cos(\phi) \tag{12}$$

$$p_y = p_\perp \sin(\phi) \tag{13}$$

$$p_z = p_\perp \sinh(\eta) \tag{14}$$

The $\hat{z}$ direction is the direction of the proton beam created by the LHC. $p_\perp$ is the component of the momentum perpendicular to the beam. ϕ is the angle between $p_\perp$ and the $\hat{x}$ direction.

The pseudorapidity coordinates and their relationships to the Cartesian coordinates are depicted in Figure 3.[11]

3 Python

3.1 Jupyter Notebooks

This exercise requires that you have access to Jupyter Notebooks to write your code. If you do not have Jupyter Notebooks on your computer, they can be download at the official website.[6]

[10] η is not technically invariant, but it is an approximation of the rapidity which is invariant so we may treat it as such.
[11] $\hat{\perp}$ is simply the direction of $p_\perp$.

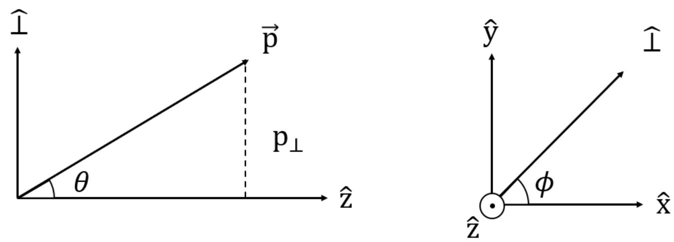

Figure 3: Two 2D views of the pseudorapidity coordinates

3.2 The Basics

This exercise requires a basic understanding of Python topics like ints, boolean conditions, lists, dictionaries, and loops. If you are unfamiliar with Python, read through the HSF Python tutorial here.[4] This tutorial will provide code segments in the form of Jupyter Notebook cells that you can follow along with to get practice. The HSF tutorial includes some topics like histogramming, scripting, and classes that, while are useful to a Python beginner, are not necessary to complete this exercise. You may skim through these parts.

3.3 Numpy

Numpy is a popular and flexible package for data analysis, linear algebra, and much more. Numpy allows us to create multi-dimensional arrays with the *ndarray* object. *Ndarrays* are useful because they work much faster than typical Python lists and Numpy functions can be used to apply operations to the entirety of the array rather than one element at a time.

You can import numpy like so:[12]

```
[ ]: import numpy as np
```

You can create an array using *np.array()* with a list as an argument like so:

```
[ ]: a = np.array([1,4,9,16,25])
```

Now, see what happens when you execute *a+5*. The line of code returns a copy of the array with five added to every single element. Now try *np.sqrt(a)*. You will see a copy of the array with the square root of each element. Also play around with functions *np.abs()*, *np.sin()*, *np.cos()*, *np.sinh()*, *np.cosh()* and see that they act on each component of the array.

Numpy can also do component-wise math with two arrays. If we define two arrays, a and b, with the same shape (number of rows and columns),[13]

[12] It is standard that Numpy is imported as np.

[13] Or generalized to higher dimensions, the same shape tuple.

corresponding components can be added, subtracted, multiplied, or divided like so:

```
[ ]: a = np.array([[1,2,3],[4,5,6]])
     b = np.array([[7,8,9],[10,11,12]])
     a+b # or a-b or a*b or a/b
```

This is different than how typical Python lists are added, where they are stitched together into a larger list. To accomplish this with ndarrays, we can use the method *np.concatenate()* with all of the lists that we want to stitch together grouped in square brackets as one argument. We can specify the *axis* which we stitch the lists together by running:

```
[ ]: np.concatenate([a,b],axis=0)
```

Notice that the end result changes when you change the axis from 0 to 1.

Another useful application of this axis in ndarrays is the ability to sort arrays along their columns or rows using *np.sort()*. To see this for yourself, create a 2D array with integer values in a mixed up order. Observe the difference between *np.sort(array)*, *np.sort(array, axis = 0)*, and *np.sort(array, axis = 1)*.

To learn more about Numpy, consult the website.[11] There, they have a *getting started* tutorial and extensive documentation on all the methods included.

3.4 Plotting

Matplotlib is a package used to plot data. All sorts of plots can be created but you will only need to know about histograms and error bar plots. For a more complete picture of what can be done with Matplotlib, refer to the Matplotlib documentation.[10] To begin, import the necessary packages.

```
[ ]: import matplotlib.pyplot as plt
     import matplotlib.text
```

matplotlib.plt will be used to draw the plot and *matplotlib.text* will allow you to label the plot and its axes.

Use Numpy to generate an array of 100 random values from 0 to 10 so that you have some data that can be plotted.

```
[ ]: data = np.random.rand(100)*10
```

The data can be plotted as a histogram using the method *plt.hist()*. In the arguments of the function, the data, number of bins, and range of the plot should be specified. The plot can then be displayed using *plt.show()*.

```
[ ]: plt.hist(data, bins=10, range=(0,10))
     #additional methods
     plt.show()
```

The additional arguments in between the creation of the histogram and its display can include labeling the plot, labeling the axes, adding text and additional graphics to the plot, the creation of a legend, and many more actions that can be found in the Matplotlib documentation.

When working with real data, there is a degree of uncertainty in all of our measurements that must be taken into account and properly portrayed. This is a feature that is lacking in the histogram, so we plot our data as a scatter plot with error bars. To begin, define parameters controlling the upper and lower bounds of the graph and the width of each bin. The reason that these are defined first is because they will be called multiple times throughout the creation of the plot and having their definitions at the beginning of that process makes it very easy to change them.

```
[ ]: x_min, x_max = 0, 10
     bin_width = 1
```

Now, you will create the bins. You will do this with two arrays, one to define the edges of the bins and one to record all of the values at the centers of the bins. The first array can be created using *np.arange()*, which requires a start value, an end value (which is not included in the array), and a step size. These are the three parameters that we defined previously. When we create a scatterplot, we want to put a point in the center of each bin; thus, we need an array to contain all of these values. To create it, simply shift the bin starting values (all of the edges except for the last one) to the right by half of a bin's width.

```
[ ]: bin_edges = np.arange(start=x_min, stop=x_max+1, step=bin_width)
     bin_centers = bin_edges[:-1] + bin_width/2
```

Next, group the data into the bins that we defined using the function *np.histogram()*. This function takes two arguments: an array of data, and a definition of the bins. Two values are returned so you must declare two variables to hold the values. However, the second value simply returns the bin edges, which has already been defined, so it is not needed. It is common in Python to name a variable that we will not use and don't care about _.

```
[ ]: counts, _ = np.histogram(data, bins=bin_edges)
```

The length of the error bars for a distribution with discrete *counts* as its y axis values can be found by using a *Poisson Distribution*. The uncertainty in the counts ν in a Poisson distribution are:

$$\delta\nu = \sqrt{\nu} \tag{15}$$

Thus, the lengths of the error bars of *data* can be found using *np.sqrt()*.

```
[ ]: count_errors = np.sqrt(counts)
```

Finally, an error bar plot can be created using *np.errorbars()*. The x-axis will be the centers of the bins, the y-axis will be the counts in each bin, and the y-errors will be the errors we found using the Poisson distribution. Setting *fmt* to *'ko'* will simply determine the format of the plot. The plot is colored black by *'k'* and set to circles with points by *'o'*. The plot can be displayed by calling *plt.show()*.

```
[ ]: plt.errorbar(x = bin_centers, y = counts, yerr = count_errors,
                  fmt = 'ko')
```

All of the previous steps should be grouped into one cell in your Jupyter Notebook. The full process for the creation of a scatterplot to display a 1D array *data* is shown below:

```
[ ]: # define initial parameters
     x_min, x_max = 0, 10
     bin_width = 1

     # define the bins of the histogram
     bin_edges = np.arange(start=x_min, stop=x_max+1, step=bin_width)
     bin_centers = bin_edges[:-1] + bin_width/2

     # histogram the data and create errors
     counts, _ = np.histogram(data, bins=bin_edges)
     count_errors = np.sqrt(counts)

     #plot the data
     plt.errorbar(x = bin_centers, y = counts, yerr = count_errors,
                  fmt = 'ko')
     # additional information
     plt.show()
```

An example of how I plot an array *masses* to display a Z-peak with all of the appropriate labels on the axes and plot is shown below. You may change this process as much as you see fit to get a plot that communicates what you want it to communicate and satisfies your aesthetic tastes.

```
[ ]: # initial parameters
x_min, x_max = 10, 160
bin_width = 1
bin_edges = np.arange(start=x_min, stop=x_max+1, step=bin_width)
bin_centers = bin_edges[:-1] + bin_width/2
counts, _ = np.histogram(masses, bins=bin_edges)
count_errors = np.sqrt(counts)

# plot
plt.errorbar(x = bin_centers, y = counts, yerr = count_errors,
             fmt = 'ko', label = 'Data', markersize = 4)
plt.title('Z Boson Signature in ATLAS Open Data')
plt.xlabel('Di-lepton mass [${m_{\mu\mu} \ or \ m_{ee}}$]',
           x=1, horizontalalignment = 'right')
plt.ylabel('Events/' + str(bin_width) + ' GeV',
           y=1, horizontalalignment = 'right')
plt.legend(frameon = False, loc = 'upper left')
plt.show()
```

3.5 Uproot

The data that ATLAS collects on the leptons that pass through it are stored in ROOT files. These files can be opened and analyzed using *Uproot*. A comprehensive introduction and documentation of uproot can be found at their website. [9]

Begin by importing *uproot*.

```
[ ]: import uproot
```

To open a file, you need to either have a root file stored to your hard drive or a url for a file stored on the internet. You will be importing our data from the ATLAS experiment using the package *atlasopenmagic*.[1] First, install the data to your computer (if you have not done so already). This can be done by executing the cell:

```
[ ]: import sys
%pip install atlasopenmagic
from atlasopenmagic import install_from_environment
install_from_environment()
```

Next, import the *atlasopenmagic* package as atom and set the available release using *atom.set_release()*. The different options for the releases can be seen by running *atom.available_releases()*.

```
[ ]: import atlasopenmagic as atom
     atom.set_release('2025e-13tev-beta')
```

Now, access a list of files that can be opened with Uproot. We first need to set the *skim*. The *skim* is essentially a filter for the type of data you want to open. You can check all your options for the skim by running *atom.available_skims()*. Now, define a list of files with the value returned by the method *atom.get_urls()*.

```
[ ]: skim = '4lep'
     files_list = atom.get_urls('data', skim, protocol = 'https',
                               cache=True)
```

To open a file, use *uproot.open()*. Let's examine the file at index zero and store this value in a variable.

```
[ ]: tree_dict = uproot.open(files_list[0])
```

ROOT files store data in what are called *TTrees*. They have various *TBranches* corresponding to different types of data collected. *tree_dict* is a dictionary of TTrees; call *tree_dict.keys()* to see the names of the different TTrees. To access a single TTree, run *tree_dict['name of the TTree']*. In the case of ATLAS open data, all choices of skim will result in a dictionary with only one TTree called *analysis*. Since we know the name of the TTree before we open the file, we can take a shortcut to open the TTree directly by running:

```
[ ]: analysis = uproot.open(files_list[0]+':analysis')
```

To see the branches of *analysis*, run *analysis.show()*.[14] If all the information is too overwhelming, you can filter for only branches with information about leptons by adding *filter_name = lep_** into the argument of the function.

Now that you have a TTree, you need to turn it into a data type that you can work with in Python. To do this, use the method *uproot.arrays()*. You must define two variables that we will use as arguments in the function. The first is an array of all the branches in the TTree that you want to analyze given whatever names you desire.[15] The second is a dictionary that connects each of the chosen names as keys to the corresponding branches as values. An example is shown below with one branch being read. Keep in mind that you can include as many attributes as there are branches on the TTree.

```
[ ]: attributes = ['n']
     aliases_dict = {'n':'lep_n'}
     leps = analysis.arrays(attributes, aliases=aliases_dict)
```

[14] If you are familiar with ROOT, think of this method as the Uproot equivalent of the TBrowser. It is less interactive but provides all the information needed about the ROOT file.

[15] It is easiest to simply remove the *'lep_'* from the branch name, so *'lep_n'* becomes simply *'n.'*

leps is a structured array managed by the Awkward Array package,[16] which is the closest thing Python has to a TTree. A structured array is divided into fields (which can be displayed with *leps.fields()*) similar to a dict. All of the data from the field *'n'* can be accessed using either *leps.n* or *leps['n']*. All of the values for each field for the first event can be accessed using *leps[0]* and likewise with subsequent events.[17]

4 Activity 1: Z Peak

4.1 Overview

In this activity, you will use all that you have learned to create a Jupyter Notebook that displays the mass of the Z bosons in ATLAS open data. To do this, you will:

1. Turn ATLAS open data into arrays that can be used in Python.
2. Create selections to remove any events that did not contain a possible Z boson.
3. Use the properties of 4-vectors to calculate the masses of the possible Z bosons.
4. Plot the masses of the possible Z bosons.

4.2 Accessing the Data

Start by importing the libraries that you need to analyze the data: numpy, uproot, matplotlib.pyplot, and matplotlib.text. Next, install and import ATLAS Open data and create a list of files with the skim *'2to4lep'*. Follow along with section 3.5 to do so. The list of files is 16 files long; not all of these are needed to view a distinct Z peak. For this activity, you will only be working with one file to reduce the processing time. In Activity 3, you will create a process to analyze all 16 files, that you can apply here.

Open the TTree called *analysis* directly like was done so in section 3.5. Next, turn *analysis* into a structured array using *uproot.arrays()*. The branches that you will need to analyze are:

- Transverse momentum (lep_pt)
- Pseudorapidity (lep_eta)

[16] Awkward Array works similar to Numpy but with less stringent rules regarding jagged arrays. Since the arrays start out initially as jagged arrays (since there are an inhomogeneous number of leptons in any given event) it is advantageous to start by working with this data type. You can change what is returned by *uproot.arrays()* with the key word argument *library*. Setting *library* equal to *'np'* will create a dictionary of numpy arrays. For those who prefer to do data analysis with *pandas*, you can set library to *'pd'* to create a dataframe.

[17] If the arrays in the fields are of higher dimension, the additional dimensions can be accessed like so: *leps[i, j, k, etc.]*

- Angle around the beam (lep_phi)
- Energy (lep_e)
- Charge (lep_charge)
- Flavor (lep_type)[18]
- Tight ID (lep_isTightID)

Remember that the first argument is a list that collects all of our chosen names for the different variables. The second argument is a dictionary that connects each of our chosen names as a key to the names of the corresponding TBranches as values. All the names in both *attributes* and *aliases_dict* must be strings. Their partial declaration is shown below. You must fill in the rest of the variables.

```
[ ]: attributes = ['pt','eta','phi', etc. ]
     aliases_dict = {'pt':'lep_pt','eta':'lep_eta', etc. }
```

Now, pass these arguments into uproot.open() to define a structured array which I will call *leps*.

4.3 Selections

Now, you will examine the charge, flavor, and tight ID of the first two leptons in each event to determine whether or not they could have come from a Z boson decay. You will create lists of boolean values, called *selections* or *cuts*, that can be applied to our array to remove all leptons that do not meet the characteristics of leptons that decayed from a Z boson.

Before creating selections, you must first sort each event in *leps* from highest to lowest transverse momentum. The two highest $p_\perp$ leptons are called the *leading* and *sub-leading* leptons and they are the leptons which will have decayed from a Z if a Z was present. To do this, you will need to create a sort of blueprint on how to sort the transverse momentum and then apply that blueprint to the whole structured array. This can be done using *np.argsort()* which returns an array of indices sorted from lowest to highest. You will need to define the axis of the function as *1* so that each event is sorted separately. The array needs to be sorted from highest to lowest a slice will need to be used to reverse the array along axis *1*. The appropriate slice is *[:,::-1]*.[19] Now that you have created your blueprint, you can apply it to all of *leps* and then remove and leptons other than the leading and subleading leptons. The whole sorting procedure, while complicated, can be completed succinctly as shown below:

[18] ROOT records the *flavor* of a lepton in the branch *lep_type*, but *type()* is already a function in Python that returns the data type of a variable. For this reason, you must call this attribute something other than *type*.

[19] The notation to reverse an array is *[::-1]*. Since we can access elements in the structured array *leps* like *leps[event #, lepton # within event]*, the slice to reverse the order of each event is *[:,::-1]*

```
[ ]: sorter = np.argsort(leps.pt,axis=1)[:,::-1]
     leps = leps[sorter][:,:2]
```

Now that you have removed all but the two highest $p_\perp$ leptons from *leps*, the selections can be created. As was discussed in section 2.3, a Z boson decays into two leptons of the same flavor and opposite charges. All charges are either $+1$ or -1, so the sum of the charges in each given event should be zero if a Z boson was present. The flavors are either 11 (electron) or 13 (muon). The value stored for flavor should be the same for the two leptons in each event with a Z boson. The *tight ID* of a given lepton is simply a boolean condition that states whether or not it can be said with certainty that a given detection was a lepton as opposed to a shower from the background. It should be true for both leptons in the event. Use this information to create 1D arrays of boolean conditions that return *True* for events that satisfy the condtion and *False* otherwise.[20]

```
[ ]: charge_cut = # charges are opposite
     flavor_cut = # flavors are the same
     tight_cut = # both are tight
```

Delay applying these cuts until you have created an array of masses. This will allow you to more easily graph the data with different combinations of the defined cuts and see how much of the data they remove.

4.4 Calculating the Mass

As was discussed in section 2.1, the mass of a particle can be calculated with:

$$m = \sqrt{E^2 - \mathbf{p} \cdot \mathbf{p}} \tag{9}$$

And, as was discussed in section 2.3, the energy and momentum of a particle undergoing decay is conserved. This means that the sum of the 4-momentum components of the leading and sub-leading leptons yields the 4-momentum components of the original Z boson. To do this, first calculate the the momentum in the x, y and z directions using the formulas from section 2.4. These can be done by using the numpy trigonometry commands like so:[21]

```
[ ]: px = leps.pt*np.cos(leps.phi)
     py = leps.pt*np.sin(leps.phi)
     pz = leps.pt*np.sinh(leps.eta)
```

[20] The charge of the first lepton in each event can be accesses with *leps.charge[:,0]*, and likewise for the flavor and tight IDs of the first leptons and all attributes of the second leptons. To combine multiple boolean arrays into a single boolean array, use the & operator instead of *and*.

[21] Note that the multiply sign '*' does not perform matrix multiplication, but component wise multiplication.

Now create the components of the 4-momentum for the di-lepton system. You must do this by adding each component for the first lepton in each event to the second lepton of each event. This can be done like so:

```
[ ]: sum_px = px[:,0] + px[:,1]
     # same for py, pz, and E
```

Now, we can find the masses of all the di-lepton systems. [22]

```
[ ]: masses = np.sqrt(np.abs(sum_e**2-sum_px**2-sum_py**2-sum_pz**2))
```

If you are both familiar with 4-vector math and are more experienced in Python I encourage you to look into the package *vector*. The documentation outlines many useful tools for physicists.[3] Using *vector*, the array of Z masses can be calculated like so:

```
[ ]: import vector
     P4 = vector.zip({'pt':leps.pt,'eta':leps.eta,
                      'phi':leps.phi,'e':leps.e})
     masses = (P4[:,0]+P4[:,1]).M
```

4.5 Plotting the Masses

Before plotting, create a cell to apply the cuts to the array of masses:

```
[ ]: masses = masses[charge_cut & flavor_cut & tight_cut]
```

Try graphing the data with and without executing this cell. The array *masses* can be reset to a pre-cut version by running the cell where it was originally created. Refer to section 3.4 to create an error bar diagram of *masses*.

5 Activity 2: Higgs Peak With Simulated Data

5.1 Overview

In this activity, you will create a Jupyter Notebook that displays the mass of the Higgs bosons in a *Monte Carlo simulation* of Higgs decays. To do this, you will:

1. Turn the file into Arrays that can be used in Python.

[22] To avoid imaginary values making it into our array of masses we have inserted absolute value bars around the radicand. This is needed because not all di-lepton systems come from a Z boson and thus not all di-lepton systems will result in a real mass.

2. Structure our data in a way to represent all possible Z boson combinations.
3. create cuts to remove all invalid Z bosons.
4. Use the properties of 4-vectors to calculate the masses of the possible Z bosons.
5. Plot the masses of the possible Z bosons.
6. Use the Z boson masses and cuts to create more cuts to remove all invalid Higgs bosons.
7. Calculate the masses of the possible Higgs bosons.
8. Plot the masses of the Higgs bosons.

5.2 accessing the data

5.2.1 Create Structured Array

Start the notebook by importing the needed packages. You can access ATLAS open data using the *atlasopenmagic* package as was done in Activity 1. Repeat the procedure outlined in section 3.5 with a couple of changes. Change the skim to *'4lep'* and when you call *atom.get_urls()* change *'data'* to *'345060'*. This is the dataset ID for the decay of a Higgs boson into 4 leptons. The complete list of dataset IDs can be found at the ATLAS Open Data website.[1] Use *uproot.open()* to open the TTree within the ROOT file. Unlike the real data, there should only be one file in the list of files to analyze.

Next, use *uproot.arrays()*, like was done in the Z boson activity, to turn the TTree into a structured array. I will once again be calling this structured array *leps*.

Now we want to sort the leptons in descending order by transverse momentum. This can be done in the exact same way it was done in the Z boson activity. The one change that you have to make is to keep four highest $p_\perp$ leptons as opposed to just the leading and sub-leading leptons.

5.2.2 Formatting the data

We want to eventually combine the data from the different combinations of the four leading leptons in each event in order to form a vector representing every possible Z boson that could have existed during each event. In the Z boson activity you could get away with combining attributes one by one, but in this activity, you want to simplify the code as much as you can. To do this, construct a 3D array to hold the data. This can be done by putting each piece of *leps* into a Numpy array declaration. Remember to first use the pseudorapidity to cartesian conversions to create 2D arrays for p_x, p_y, and p_z so that the first four components of *vecs* represent 4-momentums.

```
[ ]: # convert pt, eta, phi to px, py, pz

     # create a 3D array like so:
     lep_props = [leps.e,px,py,pz,leps.charge,leps.flavor,leps.tight]
     vecs = np.array(lep_props)
```

The 3D array holds all the data that are needed to construct all possible Z boson combinations. It can be indexed like so: *vecs[attribute, event #, lepton #]*. The attributes are indexed so that:

- *vecs[0]* would return the energy of each lepton in each event.
- *vecs[1]* the momentum in the x direction.
- *vecs[2]* the momentum in the y direction.
- *vecs[3]* the momentum in the z direction.
- *vecs[4]* the charge.
- *vecs[5]* the flavor.
- *vecs[6]* the tightID

The other indices can be used to access different kinds of information about the detector data. For example:

- *vecs[:,0]* would return all the attributes of all four leptons in the first lepton event[23] and likewise with *vecs[:,1]*, *vecs[:,2]*, etc.
- *vecs[:,:,0]* would return all the attributes of the first lepton in every event.

This 3D array could also be pictured as a 2D array of 4-momentum.[24] As can be seen in Figure 4, each horizontal, one dimensional array parallel to the one shown below is the 4-momentum of a lepton. If we were to rotate this 3D array so that the arrow were to point out of the page, we could more clearly visualize our data as a 2D array like the one shown in Figure 5.

Continue experimenting with different slices of this 3D array until you feel comfortable with what each part of it represents. Organizing the data in this way will simplify your code, however it can be ambiguous as to what you are doing since the attribute indices are not labeled like they were in the structured array.

[23] Remember that we removed all events without exactly four leptons in a given lepton event, so this is more technically the first lepton event with exactly four leptons.

[24] We don't have *true* 4-vectors here since we have added on components representing the charge, flavor, and tight ID of the lepton associated with each 4-vector but this visualization still helps.

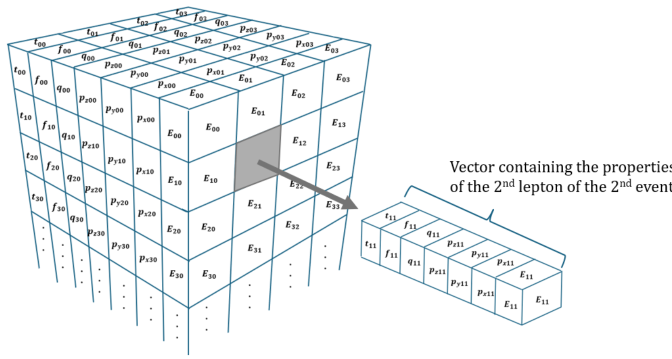

Figure 4: 3D Array with the 4 Vector for the second lepton of the second lepton event (*vecs[:,1,1]*) being removed.

$$\begin{bmatrix} \vec{P}_{00} & \vec{P}_{00} & \vec{P}_{00} & \vec{P}_{00} \\ \vec{P}_{10} & \vec{P}_{11} & \vec{P}_{12} & \vec{P}_{13} \\ \vec{P}_{20} & \vec{P}_{21} & \vec{P}_{22} & \vec{P}_{23} \\ \vdots & \vdots & \vdots & \vdots \\ \vec{P}_{n0} & \vec{P}_{n1} & \vec{P}_{n2} & \vec{P}_{n3} \end{bmatrix}$$

Figure 5: 2D array of 4-momentum

5.3 Obtaining All Possible Zs

5.3.1 Creating all possible Zs

Recall that *vecs[:,:,n]* will return a 2D array of all the attributes of the nth lepton in each event. These 2D slices of our 3D array can be added together to create six 2D arrays representing all possible dilepton systems. The combinations are:[25]

1. $Z_0 = lep_0 + lep_1$
2. $Z_1 = lep_0 + lep_2$
3. $Z_2 = lep_0 + lep_3$
4. $Z_3 = lep_1 + lep_2$
5. $Z_4 = lep_1 + lep_3$

[25]There are obviously other combinations and ways of defining Z_0 through Z_5. Changing this declarations will however change the way that H_0, H_1, and H_3 are declared later in section 5.4, as well as the "convenient order" in which Zs is declared. The specific way of defining our Z and Higgs bosons is arbitrary, what isn't arbitrary is that you absolutely can not double count leptons when you create the Higgs bosons.

6. $Z_5 = lep_2 + lep_3$

Afterwards, these arrays should be combined into a 3D array of Z bosons using *np.array()*. This can be done like so:

```
[ ]: Z0 = vecs[:,:,0] + vecs[:,:,1]
     # create Z1 through Z5
     Zs = np.array([Z0,Z1,Z2,Z5,Z4,Z3])
```

The order in which the Z bosons are "stacked" in the the 3D array *Zs* does matter. The reason is so equivalent Z bosons will be in the same indices when we create our array of Higgs bosons. This will be elaborated on further in section 5.4.

When you created *vecs*, you input an array of attributes as the argument of the constructor, but when you created *Zs* you input an array of first, second, third, etc. Z bosons in each event. For this reason, the indexing of *Zs* is different than *vecs*. *Zs* can be indexed like so: *Zs[boson #, attribute, event #]*.[26]

5.3.2 Filtering the Data

Now create the selections of the array of possible Z bosons to remove any di-lepton systems that don't exhibit Z-like behavior. To create cuts based on the values for the charge, flavor, and tight ID of the Z bosons, it must be taken into account that the vectors in the array are the vectors of a di-lepton system, rather than that of a single lepton. If the leptons are of opposite charge, then the vector for the di-lepton system will equal 0. If the flavor code for an electron is 11 and 13 for a muon, then flavor codes of 22 and 26 would signal Z-like behavior. The *tight ID* is an array of boolean conditions, and when arrays of boolean conditions are added together, they are first automatically converted to integers where *True* = 1 and *False* = 0. This means that in the *tight ID* conditions for the array *Zs*, 2 will mean both leptons satisfy *tight ID*. Go ahead and create these cuts; they should yield 2D arrays of boolean values.

5.3.3 Calculating the Masses

Calculate a 2D array of Z boson masses, *Z_mass*, using 2D slices of *Zs*.[27] Use equation 9 to do so.

5.3.4 Plotting the Masses

This step is unnecessary for creating a Higgs peak, but it will illuminate something crucial to doing so. Start by creating a cell to apply the cuts to *Z_mass*. Since it is a 2D array, you can not simply get rid of all of the non Z bosons by

[26] The boson number is indexed in the order the bosons were input into the array Zs, so *Zs[:,0]* returns Z_0, *Zs[:,1]* returns Z_1, *Zs[:,2]* Z_2, *Zs[:,3]* Z_5, *Zs[:,4]* Z_4, and *Zs[:,5]* Z_3

[27] The reason a 2D array of Z masses is needed at all is to ensure that all plotted Higgs boson in a given event has one on-shell and one off-shell Z.

typing *Zs[cuts]*.[28] Instead, you will insert a 0 at all locations where there is an invalid Z boson and later remove them once you have converted the array of masses into a 1D array. Inserting zeros can be done like so:

```
[ ]: cut = Z_charge_cut & Z_flavor_cut & Z_tight_cut
     Z_mass[np.invert(cut)] = 0
```

The selections you defined record *True* at all indices where Z-like behavior is seen. The notation in the cell defines all indices recorded *True* as 0. For these reasons, insert the function *np.invert()* in the square brackets to flip every boolean condition so that *True* now stands for di-lepton systems that do not exhibit Z-like behavior.

Now, convert the array of masses into a 1D array with *np.ndarray.flatten()* which concatenates all the rows of *Z_mass* into a single array. Afterwards, remove all masses equal to zero from the 1D array.

```
[ ]: Z_masses = np.ndarray.flatten(Z_mass)
     Z_masses = Z_masses[Z_masses != 0]
```

Next, plot an error bar plot using matplotlib. Notice two distinct peaks: one at 91 GeV as we would expect for Z bosons, and a second peak around 34 GeV. This lower peak is the off-shell Z boson peak. It is important that we see both of these peaks present because they suggest the existence of a Higgs boson decay.

Something important to note is that this data does not represent the real Z bosons that existed, but rather every possible combinations of leptons that could have resulted in a Z boson. In reality, there can only be at most two Z bosons in a given event (since we narrowed down to four leptons). More selections will narrow down our list of masses further so that all Higgs bosons we show truly existed.

5.4 Obtaining All Possible Higgs

Recall the somewhat strange order in which Z bosons were "stacked" to create *Zs*. The possible Z bosons combinations were:

1. $Z_0 = lep_0 + lep_1$
2. $Z_1 = lep_0 + lep_2$
3. $Z_2 = lep_0 + lep_3$
4. $Z_3 = lep_1 + lep_2$

[28] Well, you can, but what is returned is not a 2D array but rather a 1D array with all of the Zs removed. This technically does work for what you want to do, but it means that the next line of code, which will turn Zs into a 1D array, will no longer compile.

5. $Z_4 = lep_1 + lep_3$
6. $Z_5 = lep_2 + lep_3$

Thus, the only combinations of Z bosons that yield a Higgs boson without double counting leptons are:

1. $H_0 = Z_0 + Z_5 = (lep_0 + lep_1) + (lep_2 + lep_3)$
2. $H_1 = Z_1 + Z_4 = (lep_0 + lep_2) + (lep_1 + lep_3)$
3. $H_3 = Z_2 + Z_3 = (lep_0 + lep_3) + (lep_1 + lep_2)$

The array Zs was declared like so:

$$\begin{bmatrix} Z_0 \\ Z_1 \\ Z_2 \\ Z_5 \\ Z_4 \\ Z_3 \end{bmatrix}$$

Once split into two arrays, the corresponding Z bosons needed to form a Higgs boson are in the same array indices:

$$\begin{bmatrix} Z_0 \\ Z_1 \\ Z_2 \end{bmatrix} + \begin{bmatrix} Z_5 \\ Z_4 \\ Z_3 \end{bmatrix} = \begin{bmatrix} H_0 \\ H_1 \\ H_2 \end{bmatrix}$$

This property of corresponding Zs in the same indices will also allow us to very easily make cuts to an array of Higgs masses.

To begin, create an array *Higgs* by adding the top half of *Zs* to the bottom half. *Higgs* can be indexed in the same way that *Zs* is indexed.

```
[ ]: Higgs = Zs[:3] + Zs[3:]
```

Next, use *Z_mass* to create a cut to ensure that there is one on-shell and one off-shell Z boson per Higgs boson. Make sure that you re-execute the cell where *Z_mass* was originally defined so that this cut only selects based on the mass shells.[29] The boundaries for these cuts should be based on the Higgs boson discovery paper form 2012.[2] There, they select for an on-shell Z boson with a mass between 50 GeV and 106 GeV and an off-shell Z boson that is between 17.5 GeV and 115GeV.[30] To create the cuts, start by outlining parameters for

[29] Back when you plotted the Z masses, a zero was inserted at every index that had an invalid Z boson. If you were to use this cut version of *Z_mass* to construct *shell_cut*, then you would have the charge and flavor conditions baked into the shell condition. You want to avoid this so that each of your selections are isolated and can be applied seperately for the purposes of selection analysis.

[30] This is actually a vastly simplfied version of the actual cut that was used. The lower bound for the off-shell Z was a function of the mass of the on-shell Z. My simplified cuts simply take the widest range that was mentioned.

the upper and lower bounds of on and off-shell Z bosons, then create a boolean condition that returns true at all indices where the top and bottom halves of *Z_mass* make an on-shell off-shell pair. You will need to chain together smaller boolean conditions with the & and | [31] operators to do this.

```
[ ]: # define our mass constraints
     Z_on_min = 50
     # etc.

     # return true for all indices where a on\off shell pair exists
     shell_cut = # one of the Zs is on and the other is off
```

Next, create cuts *H_charge_cut*, *H_flavor_cut*, and *H_tight_cut* by using the & operator between the two halves of *Z_charge_cut*, *Z_flavor_cut*, and *Z_tight_cut*. What these cuts ensure is that the Higgs bosons decayed into two valid Z bosons (based on the charges, flavors, and tight IDs of the leptons which those Zs then decayed into).

5.4.1 Calculating the Masses

Calculate a 2D array of Higgs masses in the same way that the the Z masses were calculated. Observe that each column yields three of the same mass. This is because no matter the order the leptons are added, the final 4-momentum ends up the same. The only reason we are picky about the way the leptons are added was to make the *shell_cut* condition to ensure that the Higgs were made of one on-shell Z and one off-shell Z. To convert from 2D to 1D, simply take one of the rows of the 2D arrays of Higgs masses and discard the rest.

5.4.2 Plotting the Masses

First create a cell to apply the conditions to the array of Higgs masses. To do this you must convert your selections to a 1D array. Combine the conditions into a singular cut with the & operator. Then, connect the rows of each condition with | operators to flatten the selections. What this does is record *True* for every event where at least one of the possible combinations, H_0, H_1, or H_2, is a valid Higgs boson. Try different combinations of cuts to see how much background each removes from the Higgs signal.

The Higgs masses can be graphed using the same procedure as the Z masses. The only change is that the lower bound should be 100 GeV and the upper bound 160 GeV to capture the domain we expect to see the Higgs peak.[32]

[31] This is the bit-wise *or* operator.

[32] When working with real data, you will see a bunch of other interesting peaks outside of this range. Try it out in the next activity.

Bonus: the 4μ and $4e$ channels

High-energy physicists often like to add additional selections to split peaks into the portion constructed with only muons and the portion constructed with only electrons. Muons, due to their nature, produce higher resolution peaks then electrons. As an additional activity, use the array *Higgs* to create additional selections to *H_mass*. Use these selections to create plots of Higgs peaks for both channels and observe the difference in resolution.

6 Activity 3: Higgs Peak with Real Data

6.1 Overview

In this activity, you will repeat the previous analysis but this time using ATLAS open data. The procedure is the same as the previous activity.

6.2 Accessing the data

Begin by importing all the necessary packages. You can access ATLAS open data using the *atlasopenmagic* package as was done in Activity 1. Repeat the procedure outlined in section 3.5 but change the skim to *'4lep'*.

Where you can get away with only one file of real data to get a distinct Z peak, the same can not be for a Higgs peak. You must open all 16 files, convert them into structured arrays, then concatenate them together using *np.concatenate()*. This can be accomplished by first defining an empty list *leps*, then entering a for-loop where each file is turned into a structured array and concatenated to *leps* like so:

```
[ ]: leps = []
     for file in files_list:
         print(f'Analyzing file: {file}
         ({files_list.index(file)+1}/{len(files_list)})')

         # open files and turn into a structured array
         #and call it leps_partial

         leps = np.concatenate([leps,leps_partial], axis=0)
     print('Completed processing all files')
```

This cell will take a few minutes to compile. The included formatted strings will ensure that it is running properly.

6.3 Analyzing the data

The real data can be analyzed in the same way as the Monte Carlo simulated data from Activity 2.

6.4 Plotting the data

The data can be plotted in the same way as Activity 2. When all of the cuts are applied, the count of events per GeV will be low (around 15 at the Higgs peak). To match the original discovery paper, increase the bin width to 5 so that your plot displays the counts per 5 GeV (around 40 at the Higgs peak).

7 Additional Activities

There is no shortage of additional activities that you can do with Higgs bosons and ATLAS open data. The original discovery paper for the Higgs boson includes several more intricate selections that you can apply to your notebooks to get better and better Higgs signals. In addition, I encourage you to try and create a Higgs peak through a channel other than the four lepton channel. I have included a tutorial that will walk you through creating a Higgs peak through the Di-Photon channel in the references.[7] Once you feel familiar with Python, I encourage you to check out Omar Kotrach and Jacob Spiegel's tutorial on ROOT where you can create the same peaks in another programming language.[5]

Acknowledgments

Thank you to Dr. Zoya Vallari for providing me with the resources and guidance I needed to begin programming in ROOT and Python. Thank you to Omar Kotrach and Jacob Spiegel for their work on their exercise *ATLAS Open Data: Discovering the Z and Higgs Bosons*. I have based much of this exercise on their work. Thank you to Dr. Antonio Boveia who helped answer some questions I had about how the Higgs boson was discovered. We also thank everyone who proofread this exercise for errors, especially Stefan Ionascu, who was extremely helpful in his thorough review.

A special thanks goes to Osip Surdutovich for all the guidance he has given me on how to make this exercise and all the questions he has answered on how to create more intricate selections.

References

[1] ATLAS Collaboration. *ATLAS Open Magic*. `https://opendata.atlas.cern/docs/data/atlasopenmagic`. Accessed: 2026-05-24.

[2] ATLAS Collaboration. "Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC". In: *Physics Letters B* 716.1 (2012), pp. 1–29. ISSN: 0370-2693. DOI: `https://doi.org/10.1016/j.physletb.2012.08.020`. URL: `https://www.sciencedirect.com/science/article/pii/S037026931200857X`.

[3] vector contributors. *vector*. `https://pypi.org/project/vector/`. Accessed: 2026-05-24.

[4] HEP Software Foundation. *An introduction to Physics*. https://hsf-training.github.io/analysis-essentials/index.html. Accessed: 2026-05-24.

[5] Kotrach O. Spiegel J. "ATLAS Open Data: Discovering the Z and Higgs Bosons". In: *Zenodo* (2023). DOI: https://doi.org/10.5281/zenodo.10291091. URL: https://zenodo.org/records/10291091.

[6] Project Jupyter. *Jupyter*. https://jupyter.org/. Accessed: 2026-05-24.

[7] Zach Marshall. *How to rediscover the Higgs boson yourself!* https://github.com/atlas-outreach-data-tools/notebooks-collection-opendata/blob/master/13-TeV-examples/uproot_python/HyyAnalysis.ipynb. Accessed: 2025-05-25.

[8] S. Navas et al. "Review of particle physics". In: *Phys. Rev. D* 110.3 (2026), p. 030001. DOI: 10.1103/PhysRevD.110.030001. URL: https://pdg.lbl.gov/.

[9] Jim Pivarski. *Uproot*. https://uproot.readthedocs.io/en/latest/index.html. Accessed: 2026-05-24.

[10] Matplotlib development team. *Matplotlib*. https://matplotlib.org/stable/. Accessed: 2026-05-24.

[11] Numpy team. *Numpy*. https://numpy.org/. Accessed: 2026-05-24.